

Laboratorio 1 — Introducción a Wireshark

Nombre: Ricardo Godinez **Carné:** 23247 **Curso:** Redes de computadoras **Fecha:** 12/07/2026 **Entorno:** Arch Linux

1. Personalización del entorno

1.1 Creación del perfil

Perfil creado: **Ricardo_Godinez** (Edit → Configuration Profiles → +).

Ubicación en disco:

```
ls ~/.config/wireshark/profiles/
```

Evidencia — perfil activo

Profile	Type	Auto Switch	Filter
Default	Default		
Ricardo_Godinez	Personal		
<i>Bluetooth</i>	<i>Global</i>		
<i>Classic</i>	<i>Global</i>		
<i>No Reassembly</i>	<i>Global</i>		

Fig. 1 — Perfil **Ricardo_Godinez** activo.

1.2 Apertura del archivo de traza

Archivo analizado: **intro-wireshark-trace1.pcap**

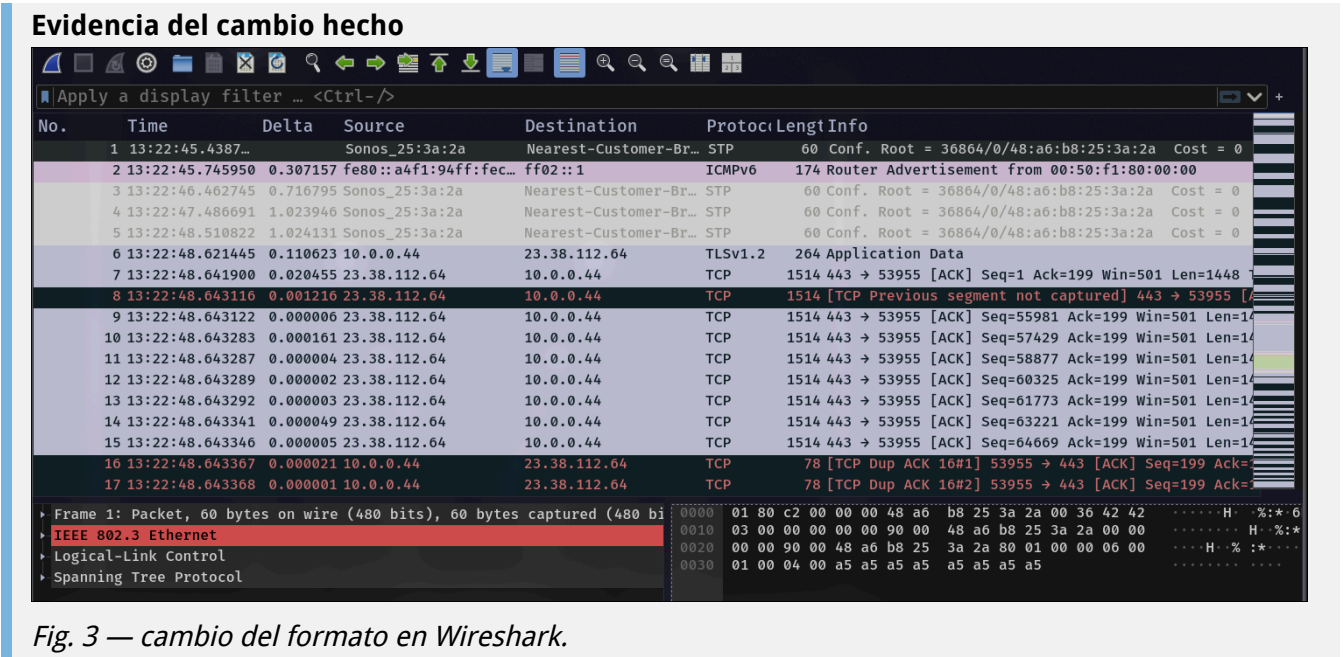
Evidencia — archivo abierto



1.3 Formato de tiempo: Time of Day

View → Time Display Format → Time of Day

Diferencia respecto al formato por defecto (*Seconds Since Beginning of Capture*):



1.4 Columna adicional: longitud del protocolo

Edit → Preferences → Columns → +

Campo	Valor usado
Title	[Proto Length]
Type	[. Custom]
Fields	[tcp.len]

Evidencia del cambio hecho

No.	Time	Delta	Source	Destination	Protocol	Length	Info	Proto Length
8	13:22:48.643116	0.001216	23.38.112.64	10.0.0.44	TCP	1514	[TCP Previous segment not captured] 443 → 53955 [ACK] Seq=545...	1448
9	13:22:48.643122	0.000006	23.38.112.64	10.0.0.44	TCP	1514	443 → 53955 [ACK] Seq=55981 Ack=199 Win=501 Len=1448 TSval=32...	1448
10	13:22:48.643283	0.000161	23.38.112.64	10.0.0.44	TCP	1514	443 → 53955 [ACK] Seq=57429 Ack=199 Win=501 Len=1448 TSval=32...	1448
11	13:22:48.643287	0.000004	23.38.112.64	10.0.0.44	TCP	1514	443 → 53955 [ACK] Seq=58877 Ack=199 Win=501 Len=1448 TSval=32...	1448
12	13:22:48.643289	0.000002	23.38.112.64	10.0.0.44	TCP	1514	443 → 53955 [ACK] Seq=60325 Ack=199 Win=501 Len=1448 TSval=32...	1448
13	13:22:48.643292	0.000003	23.38.112.64	10.0.0.44	TCP	1514	443 → 53955 [ACK] Seq=61773 Ack=199 Win=501 Len=1448 TSval=32...	1448
14	13:22:48.643341	0.000049	23.38.112.64	10.0.0.44	TCP	1514	443 → 53955 [ACK] Seq=63221 Ack=199 Win=501 Len=1448 TSval=32...	1448
15	13:22:48.643346	0.000005	23.38.112.64	10.0.0.44	TCP	1514	443 → 53955 [ACK] Seq=64669 Ack=199 Win=501 Len=1448 TSval=32...	1448
16	13:22:48.643367	0.000021	10.0.0.44	23.38.112.64	TCP	78	[TCP Dup ACK 16#1] 53955 → 443 [ACK] Seq=199 Ack=1449 Win=435...	78
17	13:22:48.643368	0.000001	10.0.0.44	23.38.112.64	TCP	78	[TCP Dup ACK 16#2] 53955 → 443 [ACK] Seq=199 Ack=1449 Win=435...	78
18	13:22:48.643368	0.000000	10.0.0.44	23.38.112.64	TCP	78	[TCP Dup ACK 16#3] 53955 → 443 [ACK] Seq=199 Ack=1449 Win=435...	78
19	13:22:48.643433	0.000065	10.0.0.44	23.38.112.64	TCP	78	[TCP Dup ACK 16#4] 53955 → 443 [ACK] Seq=199 Ack=1449 Win=435...	78
20	13:22:48.643433	0.000000	10.0.0.44	23.38.112.64	TCP	78	[TCP Dup ACK 16#5] 53955 → 443 [ACK] Seq=199 Ack=1449 Win=435...	78
21	13:22:48.643433	0.000000	10.0.0.44	23.38.112.64	TCP	78	[TCP Dup ACK 16#6] 53955 → 443 [ACK] Seq=199 Ack=1449 Win=435...	78
22	13:22:48.64343...	0.000001	10.0.0.44	23.38.112.64	TCP	78	[TCP Dup ACK 16#7] 53955 → 443 [ACK] Seq=199 Ack=1449 Win=435...	78
23	13:22:48.643434	0.000000	10.0.0.44	23.38.112.64	TCP	78	[TCP Dup ACK 16#8] 53955 → 443 [ACK] Seq=199 Ack=1449 Win=435...	78
24	13:22:48.643551	0.000117	23.38.112.64	10.0.0.44	TLV1.2	1514	Ignored Unknown Record	1448

Fig. 4 — Creacion de la nueva columna.

1.5 Eliminar / ocultar la columna Length

Evidencia del cambio hecho

No.	Time	Delta	Source	Destination	Protocol	Proto Length	Info
72	13:22:48.645527	0.000001	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=102317 Ack=199 W...
73	13:22:48.645529	0.000002	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=103765 Ack=199 W...
74	13:22:48.645531	0.000002	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=105213 Ack=199 W...
75	13:22:48.645605	0.000074	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=106661 Ack=199 W...
76	13:22:48.645610	0.000005	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=108109 Ack=199 W...
84	13:22:48.646606	0.000865	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=109557 Ack=199 W...
85	13:22:48.646613	0.000007	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=111005 Ack=199 W...
87	13:22:48.646618	0.000002	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=113901 Ack=199 W...
88	13:22:48.646622	0.000004	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=115349 Ack=199 W...
89	13:22:48.646726	0.000104	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=116797 Ack=199 W...
90	13:22:48.646731	0.000005	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=118245 Ack=199 W...
91	13:22:48.646734	0.000003	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=119693 Ack=199 W...
92	13:22:48.646738	0.000004	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=121141 Ack=199 W...
93	13:22:48.646826	0.000088	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=122589 Ack=199 W...
94	13:22:48.646846	0.000020	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=124037 Ack=199 W...
95	13:22:48.646912	0.000066	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=125485 Ack=199 W...
96	13:22:48.646918	0.000006	23.38.112.64	10.0.0.44	TCP		1448 443 → 53955 [ACK] Seq=126933 Ack=199 W...

Frame 17: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits)

Fig. 5 — Ocultar columna de longitud.

1.6 Esquema de paneles (Layout) personalizado

Evidencia del cambio hecho

No.	Time	Delta	Source	Destination	Protocol
39	13:22:48.644406	0.000029	10.0.0.44	23.38.112.64	TCP
40	13:22:48.644407	0.000001	10.0.0.44	23.38.112.64	TCP
41	13:22:48.644408	0.000001	10.0.0.44	23.38.112.64	TCP
42	13:22:48.644425	0.000017	10.0.0.44	23.38.112.64	TCP
43	13:22:48.644426	0.000001	10.0.0.44	23.38.112.64	TCP
44	13:22:48.644439	0.000013	10.0.0.44	23.38.112.64	TCP
45	13:22:48.644448	0.000009	10.0.0.44	23.38.112.64	TCP
46	13:22:48.644459	0.000011	10.0.0.44	23.38.112.64	TCP
16	13:22:48.643367	0.000021	10.0.0.44	23.38.112.64	TCP
47	13:22:48.644469	0.000010	10.0.0.44	23.38.112.64	TCP
59	13:22:48.644924	0.000063	10.0.0.44	23.38.112.64	TCP
60	13:22:48.644925	0.000001	10.0.0.44	23.38.112.64	TCP
61	13:22:48.644925	0.000000	10.0.0.44	23.38.112.64	TCP
62	13:22:48.644925	0.000000	10.0.0.44	23.38.112.64	TCP
63	13:22:48.644926	0.000001	10.0.0.44	23.38.112.64	TCP
64	13:22:48.644926	0.000000	10.0.0.44	23.38.112.64	TCP
65	13:22:48.644926	0.000000	10.0.0.44	23.38.112.64	TCP
66	13:22:48.644927	0.000001	10.0.0.44	23.38.112.64	TCP
67	13:22:48.644960	0.000033	10.0.0.44	23.38.112.64	TCP
17	13:22:48.6433...	0.000001	10.0.0.44	23.38.112.64	TCP
68	13:22:48.644960	0.000000	10.0.0.44	23.38.112.64	TCP
69	13:22:48.644961	0.000001	10.0.0.44	23.38.112.64	TCP
77	13:22:48.645739	0.000129	10.0.0.44	23.38.112.64	TCP
78	13:22:48.645739	0.000000	10.0.0.44	23.38.112.64	TCP
79	13:22:48.645740	0.000001	10.0.0.44	23.38.112.64	TCP
80	13:22:48.645740	0.000000	10.0.0.44	23.38.112.64	TCP
81	13:22:48.645740	0.000000	10.0.0.44	23.38.112.64	TCP
82	13:22:48.645741	0.000001	10.0.0.44	23.38.112.64	TCP
83	13:22:48.645741	0.000000	10.0.0.44	23.38.112.64	TCP
98	13:22:48.646952	0.000031	10.0.0.44	23.38.112.64	TCP
18	13:22:48.643368	0.000000	10.0.0.44	23.38.112.64	TCP
99	13:22:48.646953	0.000001	10.0.0.44	23.38.112.64	TCP
100	13:22:48.646953	0.000000	10.0.0.44	23.38.112.64	TCP
101	13:22:48.646954	0.000001	10.0.0.44	23.38.112.64	TCP
102	13:22:48.646954	0.000000	10.0.0.44	23.38.112.64	TCP

Frame 17: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits)

Internet II, Src: Apple_98:d9:27 (78:4f:43:98:d9:27), Dst: Maxlinear_80:00:00 (00:50:f1:80:00:00)

Internet Protocol Version 4, Src: 10.0.0.44, Dst: 23.38.112.64

Transmission Control Protocol, Src Port: 53955, Dst Port: 443, Seq: 199, Ack: 1449, Len: 0

Source Port: 53955

Destination Port: 443

[Stream index: 0]

[Stream Packet Number: 12]

[Conversation completeness: Incomplete (8)]

[TCP Segment Len: 0]

Sequence Number: 199 (relative sequence number)

Sequence Number (raw): 1356859536

[Next Sequence Number: 199 (relative sequence number)]

Acknowledgment Number: 1449 (relative ack number)

Acknowledgment number (raw): 1565891345

1011 = Header Length: 44 bytes (11)

.... 0000 0001 0000 = Flags: 0x010 (ACK)

Window: 4358

0000 00 50 f1 80 00 00 78 4f 43 98 d9 27 08 00 45 00 P...XO C...E

0010 00 40 00 00 40 00 40 06 a9 26 0a 00 00 2c 17 26 @.d..8...6

0020 70 40 d2 c3 01 bb 50 e0 08 90 5d 55 9b 11 b0 10 p@...P...JU...

0030 11 06 43 f7 00 00 01 01 08 0a 0d b7 68 cb c3 a1 .C.....h...

0040 5e c5 01 01 05 0a 5d 56 6a 6d 5d 56 75 bd ^.....]V jmVu

Fig. 6 — Cambio de layout.

1.7 Regla de color: TCP con bandera SYN = 1

Evidencia — regla de color aplicada

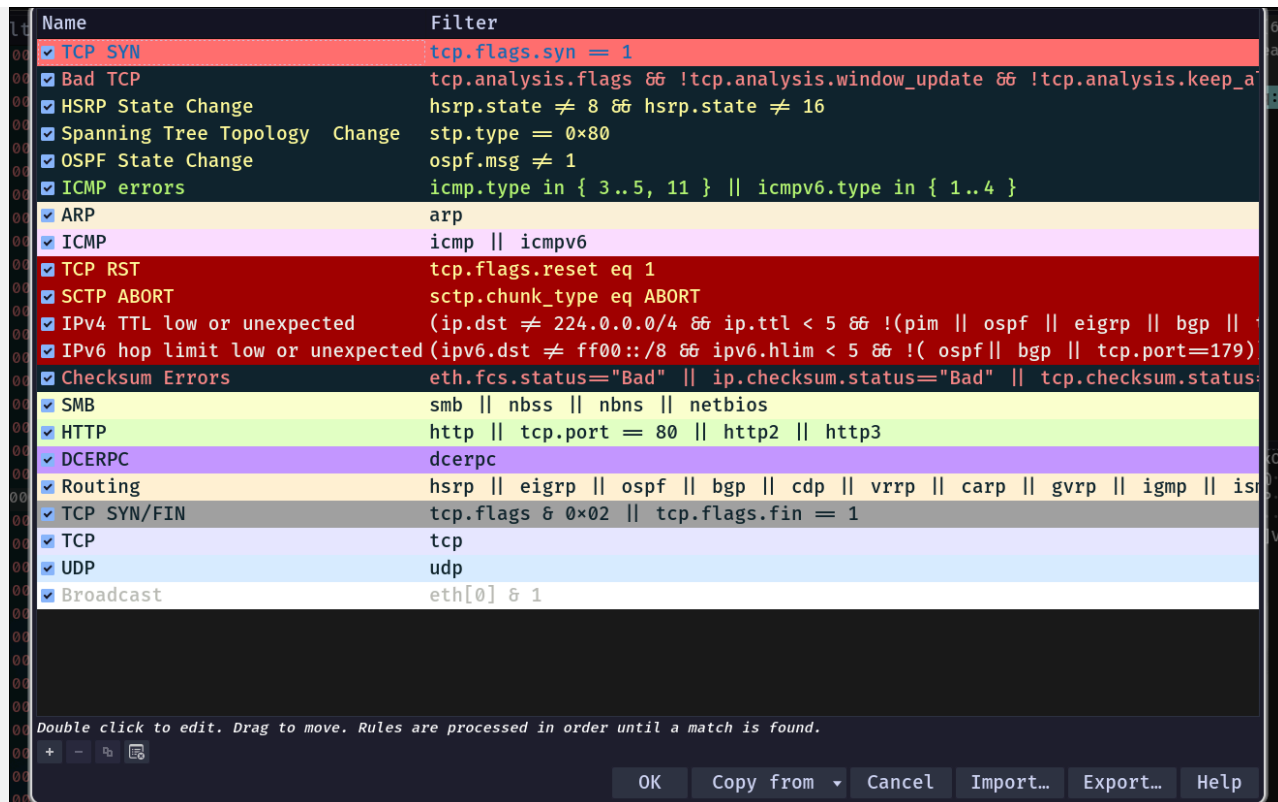


Fig. 7 — Configuración de la regla de color.

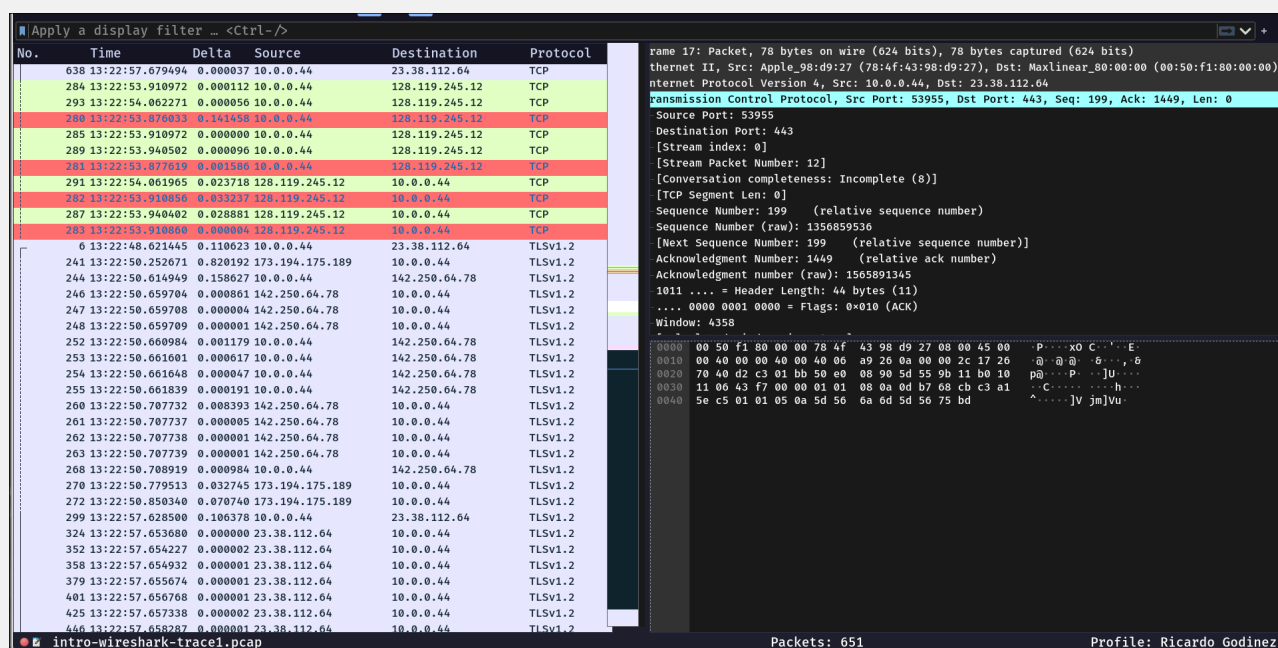
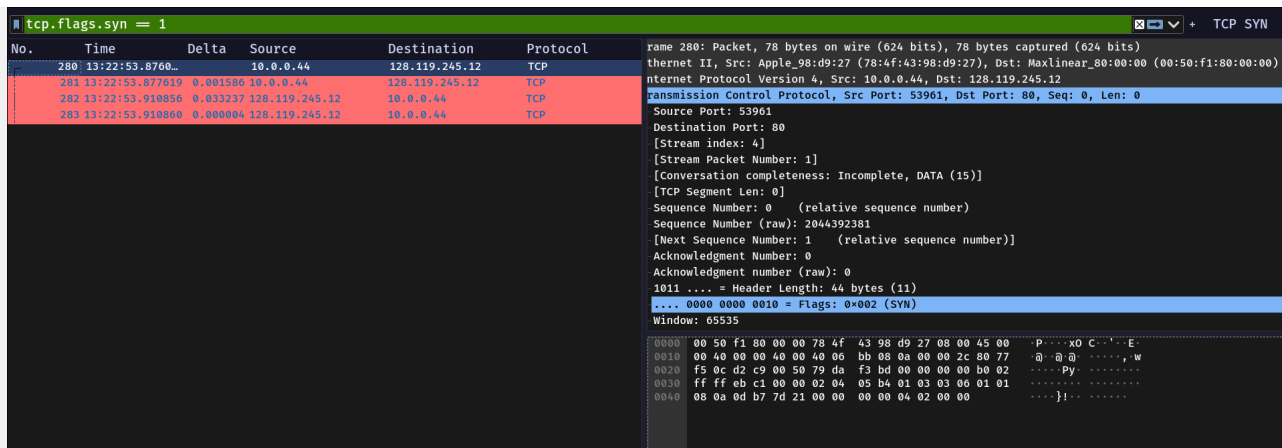


Fig. 8 — Paquetes TCP SYN resaltados con el color definido.

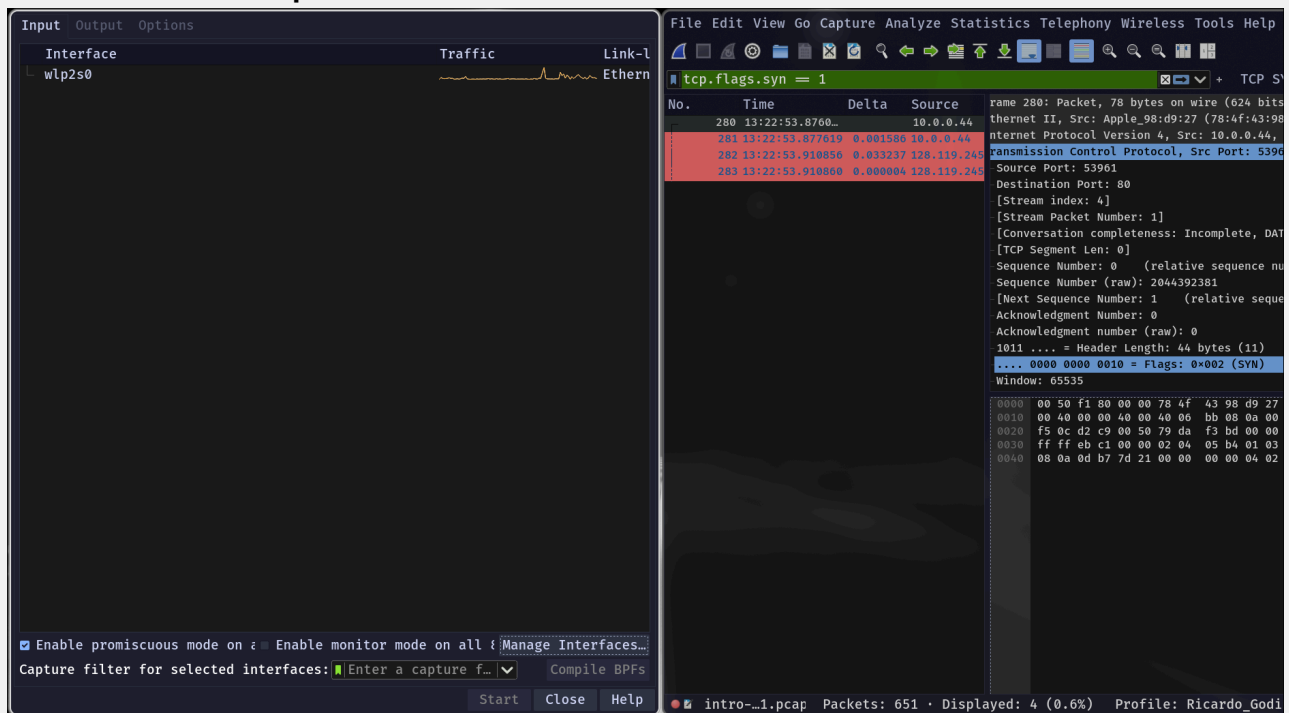
1.8 Botón de filtro (Filter Button)

Evidencia — botón funcionando



1.9 Ocultar interfaces virtuales

Evidencia — lista simplificada de interfaces



1.10 Entorno final personalizado

Evidencia — vista global

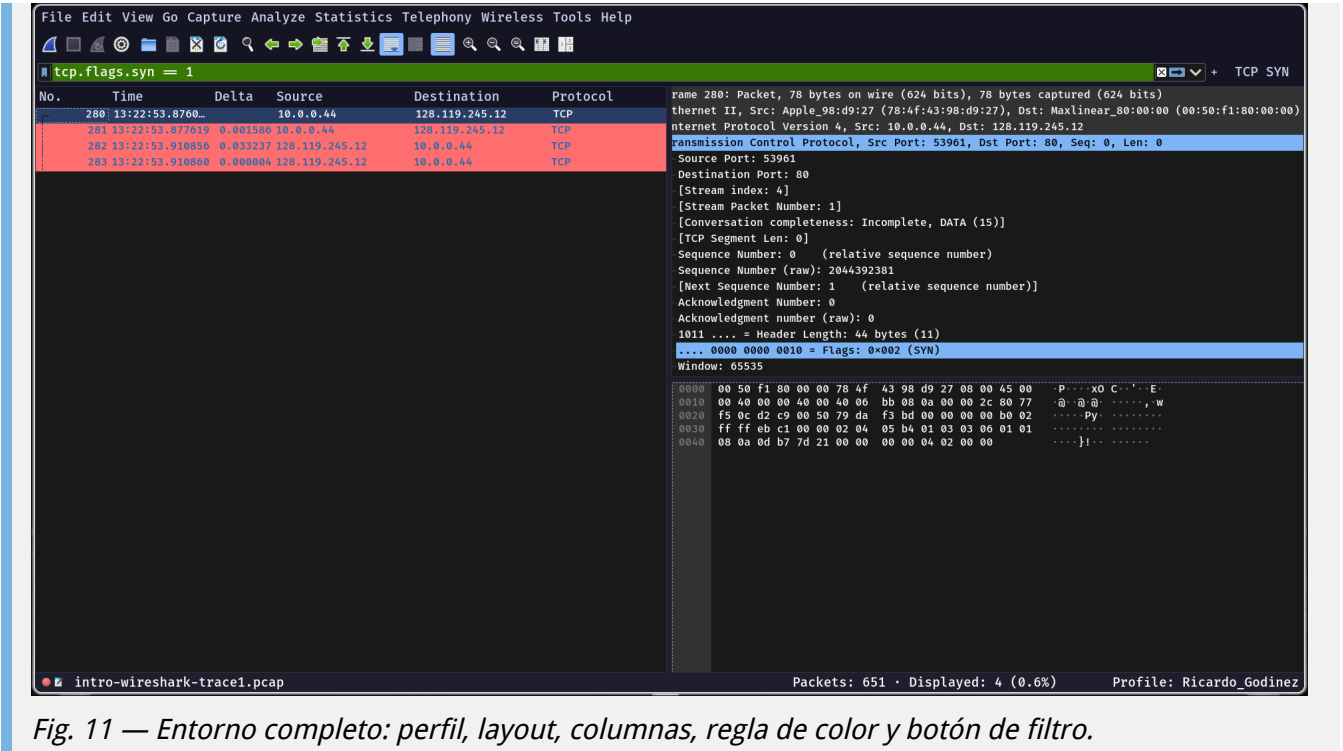


Fig. 11 — Entorno completo: perfil, layout, columnas, regla de color y botón de filtro.

2. Configuración de la captura de paquetes (ring buffer)

2.1 Salida de `ip/ifconfig`

Salida obtenida:

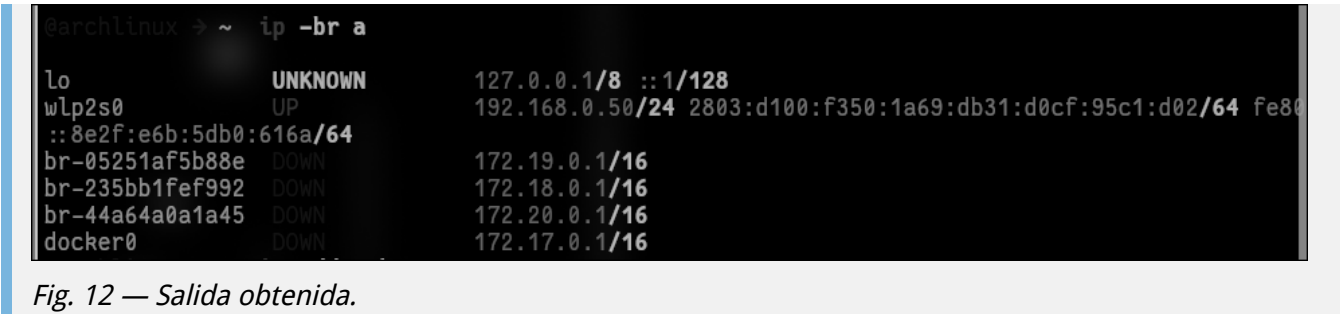


Fig. 12 — Salida obtenida.

Explicación de lo observado:

El sistema reporta **6 interfaces**, número que coincide exactamente con el indicador "6 interface(s)" que Wireshark mostraba en su pantalla de bienvenida antes de aplicar el filtrado. De estas, **solo una es física**.

#	Interfaz	Tipo	Estado	Dirección(es)	MTU	Descripción
1	lo	Virtual (kernel)	UNKNOWN	127.0.0.1/8, ::1/128	65536	Loopback. Tráfico interno del host; nunca sale a la red.
2	wlp2s0	Física (WiFi)	UP	192.168.0.50/24, IPv6 global y link-local	1500	Única interfaz con conectividad real. MAC 14:5a:fc:2d:83:8f.
3	br-05251af5b88e	Virtual (bridge)	DOWN	172.19.0.1/16	1500	Bridge de red custom de Docker Compose.

#	Interfaz	Tipo	Estado	Dirección(es)	MTU	Descripción
4	br-235bb1fef992	Virtual (bridge)	DOWN	172.18.0.1/16	1500	Bridge de red custom de Docker Compose.
5	br-44a64a0a1a45	Virtual (bridge)	DOWN	172.20.0.1/16	1500	Bridge de red custom de Docker Compose.
6	docker0	Virtual (bridge)	DOWN	172.17.0.1/16	1500	Bridge por defecto de Docker.

2.2 Desactivación de interfaces virtuales

Evidencia

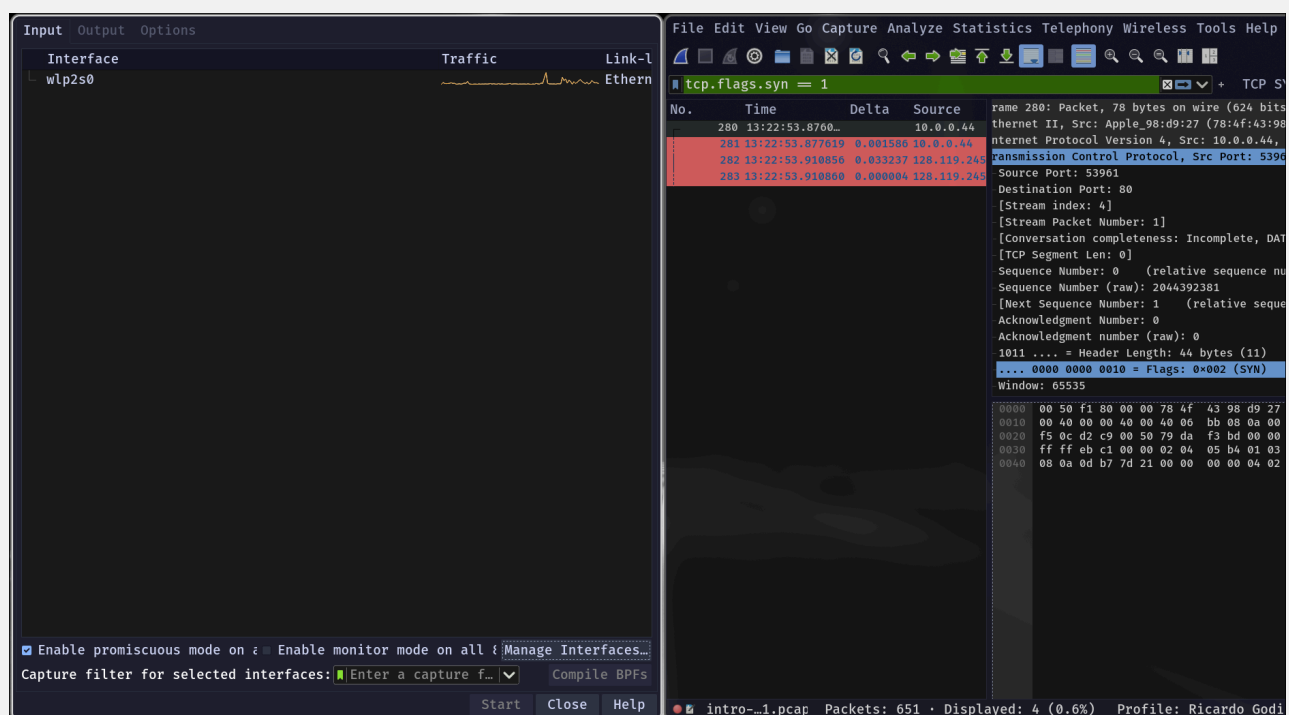


Fig. 9 — Solo la interfaz [WiFi/Ethernet] queda habilitada.

2.3 Configuración del ring buffer

Capture to a permanent file

File:

Output format: ☒ pcapng ☐ pcap

Compression: ☒ None ☐ gzip ☐ LZ4

☒ Create a new file automatically...

☐ after	100000	- +	packets
☒ after	5	- +	megabytes ▾
☐ after	1	- +	seconds ▾
☐ when time is a multiple of	1	- +	hours ▾

File infix pattern

☒ YYYYmmDDHHMMSS_NNNNN

☐ NNNNN_YYYYmmDDHHMMSS

☒ Use a ring buffer with ▾ files

Fig. 13 — Pestaña Output con la configuración de ring buffer.

2.4 Generación de tráfico y archivos creados

Tráfico generado mediante: ping en modo *flood* con paquetes de tamaño ampliado, dirigido al resolver público de Google:

```
sudo ping -f -s 1400 8.8.8.8
```

Evidencia — archivos del ring buffer


```

searchlinux ~ ls -lh ~/lab1_23247*
-rw-r----- ricgo users 4.8 MB Sun Jul 12 20:20:45 2026  /home/ricgo/lab1_23247_20260712202044_00108.pcap
-rw-r----- ricgo users 4.8 MB Sun Jul 12 20:20:46 2026  /home/ricgo/lab1_23247_20260712202045_00109.pcap
-rw-r----- ricgo users 4.8 MB Sun Jul 12 20:20:46 2026  /home/ricgo/lab1_23247_20260712202046_00110.pcap
-rw-r----- ricgo users 4.8 MB Sun Jul 12 20:20:47 2026  /home/ricgo/lab1_23247_20260712202046_00111.pcap
-rw-r----- ricgo users 4.8 MB Sun Jul 12 20:20:48 2026  /home/ricgo/lab1_23247_20260712202047_00112.pcap
-rw-r----- ricgo users 4.8 MB Sun Jul 12 20:20:53 2026  /home/ricgo/lab1_23247_20260712202048_00113.pcap
-rw-r----- ricgo users 4.8 MB Sun Jul 12 20:20:54 2026  /home/ricgo/lab1_23247_20260712202053_00114.pcap
-rw-r----- ricgo users 4.8 MB Sun Jul 12 20:20:56 2026  /home/ricgo/lab1_23247_20260712202054_00115.pcap
-rw-r----- ricgo users 4.8 MB Sun Jul 12 20:22:25 2026  /home/ricgo/lab1_23247_20260712202056_00116.pcap
-rw-r----- ricgo users 174 KB Sun Jul 12 20:23:15 2026  /home/ricgo/lab1_23247_20260712202225_00117.pcap

```

Fig. 11 — Archivos `lab1_[carné]_0000N_*.pcap` creados por el ring buffer.

Archivos adjuntos a la entrega:

Ver los archivos en [files/](#)

3. Análisis de paquetes — Protocolo HTTP

3.1 Procedimiento

Evidencia — captura HTTP

The screenshot displays the Wireshark network protocol analyzer interface. The top pane shows a list of captured packets, with the selected packet being an HTTP GET request for `/favicon.ico`. The middle pane shows the details of the selected packet, including the HTTP status code `200 OK`. The bottom pane shows the raw data of the packet in hexadecimal and ASCII format.

Fig. 15 — Paquetes HTTP filtrados. Se observan el `GET` y la respuesta `200 OK`.

3.2 Respuestas

a) ¿Qué versión de HTTP está ejecutando su navegador?

HTTP/1.1

La versión se declara al final de la línea de solicitud del paquete 134:

```
GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1
Host: gaia.cs.umass.edu
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/150.0.0 Safari/537.36
```

De paso, el User-Agent delata bastante: navegador basado en Chromium corriendo sobre Linux x86_64.

b) ¿Qué versión de HTTP está ejecutando el servidor?

HTTP/1.1

La versión aparece al inicio de la línea de estado del paquete 152:

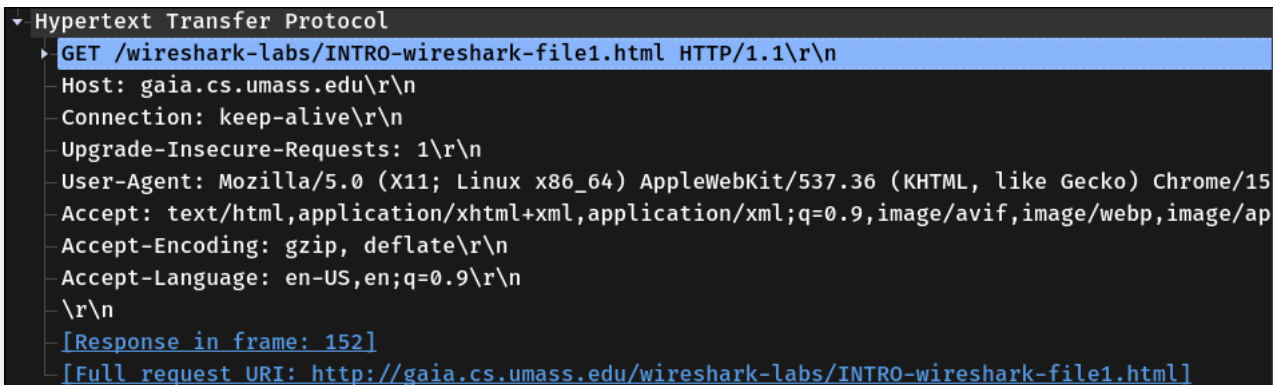
```
HTTP/1.1 200 OK
Date: Mon, 13 Jul 2026 03:08:15 GMT
Server: Apache/2.4.62 (AlmaLinux) OpenSSL/3.5.5 mod_fcgid/2.3.9
mod_perl/2.0.12 Perl/v5.32.1
Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT
ETag: "51-64231b6715777"
Accept-Ranges: bytes
Content-Length: 81
Content-Type: text/html
```

Cliente y servidor coinciden en HTTP/1.1, así que la conversación fluye sin problema.

c) ¿Qué lenguajes indica el navegador que acepta al servidor?

Inglés — concretamente en-US, en; q=0.9

Esto sale del encabezado **Accept-Language** que el navegador manda dentro del **GET** (paquete 134). Ahí le está diciendo al servidor en qué idiomas prefiere recibir la página, y en qué orden:

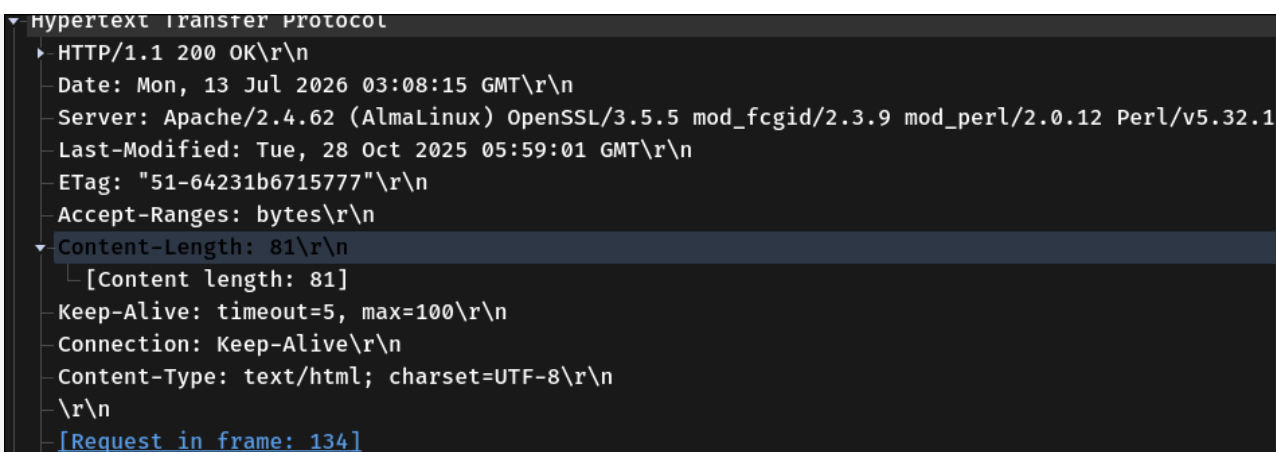


```
▼ Hypertext Transfer Protocol
  ▶ GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1\r\n
  Host: gaia.cs.umass.edu\r\n
  Connection: keep-alive\r\n
  Upgrade-Insecure-Requests: 1\r\n
  User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/15
  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/ap
  Accept-Encoding: gzip, deflate\r\n
  Accept-Language: en-US,en;q=0.9\r\n
  \r\n
  [Response in frame: 152]
  [Full request URI: http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html]
```

Fig. 16 — Header *Accept-Language*.

d) ¿Cuántos bytes de contenido fueron devueltos por el servidor?

Respuesta: 81 bytes



```
▼ Hypertext Transfer Protocol
  ▶ HTTP/1.1 200 OK\r\n
  Date: Mon, 13 Jul 2026 03:08:15 GMT\r\n
  Server: Apache/2.4.62 (AlmaLinux) OpenSSL/3.5.5 mod_fcgid/2.3.9 mod_perl/2.0.12 Perl/v5.32.1
  Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT\r\n
  ETag: "51-64231b6715777"\r\n
  Accept-Ranges: bytes\r\n
  Content-Length: 81\r\n
  [Content length: 81]
  Keep-Alive: timeout=5, max=100\r\n
  Connection: Keep-Alive\r\n
  Content-Type: text/html; charset=UTF-8\r\n
  \r\n
  [Request in frame: 134]
```

Fig. 17 — Bytes de contenido devueltos por el servidor.

e) Ante un problema de rendimiento en la descarga, ¿en qué elementos de la red convendría "escuchar" los paquetes? ¿Es conveniente instalar Wireshark en el servidor? Justifique.

El asunto es que un cuello de botella puede estar en cualquier punto del camino entre el cliente y el servidor. Si uno captura solo en un extremo, ve los síntomas pero no sabe de dónde vienen.

Lo correcto es capturar en **varios puntos a la vez** y comparar. El lugar donde los síntomas aparecen por primera vez es el que delata dónde está el problema.

Podemos poner varios observadores y en orden de donde es mas conveniente ponerlos es:

- En nuestra pc
- En el router de salida
- En un switch
- En el balanceador
- En el server

La verdad no es conveniente instalar Wireshark en el servidor, aunque pueda parecer útil consume muchos recursos, si la ponemos en un servidor que sospechamos que esta lento va a estar mucho peor.

Ademas de que wireshark es una herramienta grafica, cosa que los servidores no tienen ese entorno habitualmente. Hay herramientas mas adecuadas para este caso como tcpdump, este solo graba los paquetes en un archivo y casi no consume recursos.

4. Discusión

4.1 Sobre la primera parte

La primera parte me parecio bastante interactiva e interesante, talvez un poco complicada el ponernos de acuerdo para el uso del codigo morse porque era muy dificl poder verificar bien que se queria decir y si se queria saber preciso requeria mucho tiempo. Que el trabajo sea grupal me gusto pero talvez algo mas dinamico no tan pausado.

4.2 Sobre la personalizacion de wireshark

Me gusto que se pudiera personalizar varias cosas de esta herramienta, desde los colores, filtros, cambiar la interfaz... que hacen que la experiencia usandolo sea mas sencilla, y mas facil.

4.3 Sobre usar wireshark

Al principio fue difcil entender lo que estaba pasado, porque aunque despues se podia personalizar cuando se abre la herramienta la primera vez se veia confusa, y me costo un poco entender todo lo que estaba pasando.

4.4 Dificultades y aprendizajes

Como mencione lo que se me dificulto fue comprender todo lo que ofrecia la herramienta y que es lo que estabamos midiendo. Luego entendi que es como ver todas las partes de los paquetes destripados y ver como se comunican entre ellos.

5. Conclusiones

1. Wireshark es una herramienta muy util para ver el trafico de la red y verla completa.
2. El ring buffer es una herramienta muy util para ver todo el ciclo de los paquetes y poder detectar errores.
3. Es importante los protocolos como HTTPS porque es mucho mas seguro que HTTP que nos da info de todo.